

# Use Case 2: Healthcare Patient Data Integration

---

## Batch Processing with Snowflake + PySpark + AWS Glue

---

### 1. Overview

---

This use case focuses on building a batch data integration pipeline for a healthcare organization. You will ingest patient data from multiple source systems in different formats (CSV and JSON), perform data cleansing and transformation using PySpark on AWS Glue, mask sensitive patient information (PII), and load the unified data into Snowflake across a three-layer architecture (Raw, Staging, Curated).

**Technology Stack:** - **Source Data:** CSV files (EHR exports, pharmacy records), JSON Lines (diagnosis feeds) - **Processing:** AWS Glue with PySpark - **Storage/Analytics:** Snowflake (Raw / Staging / Curated schemas) - **Key Techniques:** JSON flattening, PII masking (SHA-256), null handling, deduplication, incremental loads with Glue bookmarks

---

### 2. Business Scenario

---

**MedCare Hospital** is a mid-sized healthcare provider that has patient data scattered across three independent systems:

1. **Electronic Health Records (EHR)** -- Exports patient demographics as CSV files nightly.
2. **Clinical Diagnosis System** -- Streams diagnosis records as JSON Lines feeds, including nested physician information.
3. **Pharmacy System** -- Generates prescription records as CSV files.

The hospital's analytics team needs a unified, cleansed, and privacy-compliant patient data warehouse to support:

- Patient 360-degree views for care coordination
- Population health analytics (diagnosis trends, prescription patterns)
- Regulatory compliance (HIPAA) via PII masking
- Operational dashboards for physician workload and drug utilization

Your task is to build the ETL pipeline that integrates these three sources, handles data quality issues, masks Protected Health Information (PHI/PII), and loads the results into Snowflake for downstream analytics.

---

### 3. Learning Objectives

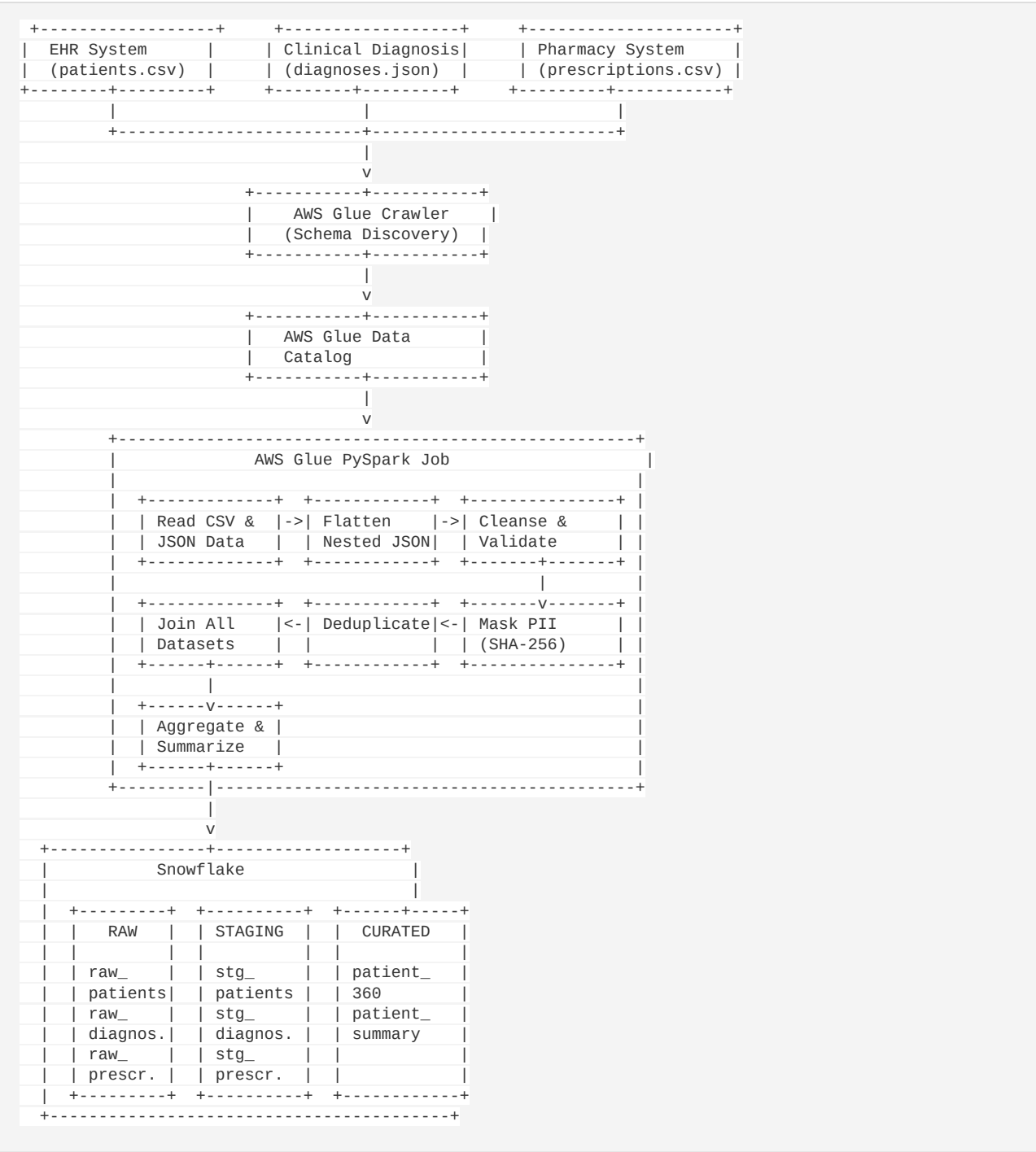
---

By completing this use case, you will be able to:

- [ ] Ingest and process multiple data formats (CSV and JSON Lines) in PySpark
- [ ] Flatten nested JSON structures (e.g., physician objects inside diagnosis records)
- [ ] Implement PII/PHI masking using SHA-256 hashing on sensitive fields (SSN, patient names)
- [ ] Handle null values with appropriate strategies per field (fill, drop, flag)
- [ ] Detect and remove duplicate records

- [ ] Validate and standardize date fields
- [ ] Join multiple datasets to create unified patient views
- [ ] Build aggregated summary tables for analytics
- [ ] Configure AWS Glue job bookmarks for incremental data processing
- [ ] Load data into Snowflake's three-layer architecture (Raw, Staging, Curated)

#### 4. Architecture Diagram



#### 5. Dataset Descriptions

5.1 patients.csv (80 records)

| Column             | Data Type | Description                  |  |
|--------------------|-----------|------------------------------|--|
| patient_id         | STRING    | Unique patient identifier    |  |
| first_name         | STRING    | Patient first name           |  |
| last_name          | STRING    | Patient last name            |  |
| date_of_birth      | DATE      | Patient date of birth        |  |
| gender             | STRING    | Patient gender               |  |
| ssn                | STRING    | Social Security Number       |  |
| city               | STRING    | Patient city of residence    |  |
| state              | STRING    | Patient state (abbreviation) |  |
| insurance_provider | STRING    | Insurance company name       |  |

5.2 diagnoses.json (120 records -- JSON Lines format)

Each line is a standalone JSON object. Note the **nested physician object**.

| Field | Data Type | Description |  |
|-------|-----------|-------------|--|
|-------|-----------|-------------|--|

|                     |        |                                      |  |
|---------------------|--------|--------------------------------------|--|
| diagnosis_id        | STRING | Unique diagnosis identifier          |  |
| patient_id          | STRING | Foreign key to patients              |  |
| diagnosis_code      | STRING | ICD-10 diagnosis code                |  |
| description         | STRING | Human-readable diagnosis description |  |
| diagnosis_date      | DATE   | Date diagnosis was recorded          |  |
| physician           | OBJECT | Nested physician information         |  |
| physician.id        | STRING | Physician identifier                 |  |
| physician.name      | STRING | Physician full name                  |  |
| physician.specialty | STRING | Medical specialty                    |  |

Sample record:

```
{
  "diagnosis_id": "DIAG001",
  "patient_id": "PAT001",
  "diagnosis_code": "E11.9",
  "description": "Type 2 diabetes mellitus without complications",
  "diagnosis_date": "2024-03-15",
  "physician": {
    "id": "DR001",
    "name": "Dr. Sarah Mitchell",
    "specialty": "Endocrinology"
  }
}
```

5.3 prescriptions.csv (100 records)

| Column                   | Data Type | Description                    | Notes                                    |
|--------------------------|-----------|--------------------------------|------------------------------------------|
| prescription_id          | STRING    | Unique prescription identifier | Format: RX001-RX100                      |
| patient_id               | STRING    | Foreign key to patients        | Links to patients.csv                    |
| drug_name                | STRING    | Name of prescribed medication  | E.g., Metformin, Lisinopril, Amoxicillin |
| dosage                   | STRING    | Drug dosage                    | E.g., 500mg, 10mg; some are empty        |
| frequency                | STRING    | How often to take the drug     | E.g., Once daily, Twice daily, As needed |
| start_date               | DATE      | Prescription start date        | Format: YYYY-MM-DD                       |
| end_date                 | DATE      | Prescription end date          | NULL if ongoing prescription             |
| prescribing_physician_id | STRING    | Foreign key to physician       | Format: DR001-DR012                      |

## 6. Tasks

### Task 1: Set Up Snowflake Environment

Create the Snowflake database and schema structure to support the three-layer architecture.

**Requirements:** - Create database MEDCARE\_DW - Create three schemas: RAW, STAGING, CURATED - Create raw layer tables that mirror the source data (no transformations) - Create staging layer tables with masked PII fields and flattened structures - Create curated layer tables for unified patient views and aggregated summaries - Set up appropriate data types, especially for dates and masked hash fields

**Key considerations:** - Raw tables should accept data as-is from source (use VARCHAR for flexibility) - Staging tables should have proper data types and constraints - Curated tables should be optimized for analytical queries

### Task 2: Read CSV and JSON Data in PySpark

Load all three source datasets into PySpark DataFrames.

**Requirements:** - Read patients.csv with header and inferred schema - Read prescriptions.csv with header and inferred schema - Read diagnoses.json as JSON Lines (one JSON object per line) - Print schemas and sample rows for verification

**Key considerations:** - For CSV files, handle the header row correctly - For JSON, ensure the nested physician object is read as a struct type - Verify record counts match expectations (80 patients, 120 diagnoses, 100 prescriptions)

---

### Task 3: Flatten Nested JSON (Physician Object)

Extract the nested physician information from the diagnoses dataset into flat columns.

**Requirements:** - Extract `physician.id` as `physician_id` - Extract `physician.name` as `physician_name` - Extract `physician.specialty` as `physician_specialty` - Drop the original nested physician column - Handle records where the physician object is null

**Key PySpark functions:** - `col("physician.id")` or `col("physician").getField("id")` - `withColumn()` to add new flat columns - `drop()` to remove the nested column

---

### Task 4: Handle Nulls, Duplicates, and Invalid Dates

Clean the data by addressing quality issues across all three datasets.

**Requirements:** - **Null handling:** - For `diagnosis_date`: Flag null dates with a default value (e.g., "1900-01-01") and add an `is_date_missing` boolean column - For `description`: Replace nulls with "Description not available" - For `end_date` in prescriptions: Leave as null (indicates ongoing prescription) - For physician fields: Replace nulls with "UNKNOWN" - **Deduplication:** - Remove exact duplicate rows across all datasets - For patients, deduplicate on `patient_id` keeping the first occurrence - **Date validation:** - Ensure all dates are in YYYY-MM-DD format - Flag any dates in the future as invalid

---

### Task 5: Mask PII Using SHA-256

Apply irreversible hashing to sensitive patient information for HIPAA compliance.

**Requirements:** - Mask `ssn` field using SHA-256 hash - Mask `first_name` and `last_name` using SHA-256 hash - Keep original `patient_id` as the linking key (not PII) - Store masked values in staging tables

**Key PySpark function:**

```
from pyspark.sql.functions import sha2

df = df.withColumn("ssn_masked", sha2(col("ssn"), 256))
df = df.withColumn("first_name_masked", sha2(col("first_name"), 256))
df = df.withColumn("last_name_masked", sha2(col("last_name"), 256))
```

---

### Task 6: Join Patients + Diagnoses + Prescriptions

Create a unified patient dataset by joining all three cleansed sources.

**Requirements:** - Join patients with diagnoses on `patient_id` (left join -- keep all patients) - Join the result with prescriptions on `patient_id` (left join) - Handle patients with no diagnoses or no prescriptions - Handle patients with multiple diagnoses and/or prescriptions (expect row expansion) - Select relevant columns for the `patient_360` curated table

---

### Task 7: Create Patient Summary Aggregation

Build a summary table with one row per patient containing aggregated metrics.

**Requirements:** - Count of diagnoses per patient - Count of active prescriptions (where `end_date` is null) - Count of total prescriptions - List of distinct diagnosis codes (as a comma-separated string) - List of distinct drug names (as a comma-separated

string) - Most recent diagnosis date - Most recent prescription start date

**Key PySpark functions:** - `groupBy()`, `agg()` - `count()`, `countDistinct()` - `collect_set()`, `concat_ws()` - `max()`

---

## Task 8: Configure Glue Bookmarks for Incremental Processing

Set up AWS Glue job bookmarks to enable incremental data processing.

**Requirements:** - Enable job bookmarks in the Glue job configuration - Use `transformation_ctx` parameter when reading data sources - Commit the bookmark after successful processing - Understand how bookmarks track which data has already been processed

**Key Glue APIs:**

```
from awsglue.context import GlueContext
from awsglue.job import Job

job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# Read with bookmark tracking
datasource = glueContext.create_dynamic_frame.from_catalog(
    database="medcare_db",
    table_name="patients",
    transformation_ctx="patients_ctx"
)

# At the end
job.commit()
```

---

## Task 9: Load Data to Snowflake Across Three Layers

Write the processed data to Snowflake's Raw, Staging, and Curated schemas.

**Requirements:** - **Raw layer:** Load source data as-is (no transformations) - **Staging layer:** Load cleansed, masked, and flattened data - **Curated layer:** Load the unified `patient_360` view and `patient_summary` aggregation - Use the Snowflake Spark connector for writing - Configure connection properties (account, warehouse, database, schema)

**Snowflake Spark connector options:**

```
sfOptions = {
    "sfURL": "<account>.snowflakecomputing.com",
    "sfUser": "<username>",
    "sfPassword": "<password>",
    "sfDatabase": "MEDCARE_DW",
    "sfSchema": "RAW",
    "sfWarehouse": "MEDCARE_WH"
}

df.write \
    .format("net.snowflake.spark.snowflake") \
    .options(**sfOptions) \
    .option("dbtable", "RAW_PATIENTS") \
    .mode("append") \
    .save()
```

---

## 7. Expected Outputs

### Snowflake Tables

| Schema  | Table Name        | Description                                        | Approx Rows |
|---------|-------------------|----------------------------------------------------|-------------|
| RAW     | raw_patients      | Source patient data, no transformations            | 80          |
| RAW     | raw_diagnoses     | Source diagnosis data (JSON flattened for storage) | 120         |
| RAW     | raw_prescriptions | Source prescription data, no transformations       | 100         |
| STAGING | stg_patients      | Cleansed patients with masked PII                  | 80          |
| STAGING | stg_diagnoses     | Flattened, cleansed diagnoses                      | 120         |
| STAGING | stg_prescriptions | Cleansed prescriptions with null handling          | 100         |
| CURATED | patient_360       | Unified patient-diagnosis-prescription view        | ~200+       |
| CURATED | patient_summary   | One row per patient with aggregated metrics        | 80          |

Sample Queries on Curated Data

```
-- Top 10 most common diagnoses
SELECT diagnosis_code, description, COUNT(*) as diagnosis_count
FROM CURATED.patient_360
GROUP BY diagnosis_code, description
ORDER BY diagnosis_count DESC
LIMIT 10;

-- Patients with more than 3 active prescriptions
SELECT patient_id, active_prescription_count, diagnosis_count
FROM CURATED.patient_summary
WHERE active_prescription_count > 3
ORDER BY active_prescription_count DESC;

-- Insurance provider distribution
SELECT insurance_provider, COUNT(DISTINCT patient_id) as patient_count
FROM CURATED.patient_360
GROUP BY insurance_provider
ORDER BY patient_count DESC;
```



---

## 8. Hints

---

### Level 1 -- Getting Started (Structural Guidance)

- Start by reading all three files and printing their schemas. For JSON, PySpark will auto-detect the nested structure.
- To flatten the physician struct, use `col("physician.id").alias("physician_id")` in a `select()` statement.
- For SHA-256 masking, import `sha2` from `pyspark.sql.functions` and apply it with `withColumn`.
- Use `df.na.fill()` for filling nulls and `df.dropDuplicates()` for removing duplicates.

### Level 2 -- Intermediate Help (Logic Guidance)

- When joining all three datasets, use left joins to preserve patients who may not have diagnoses or prescriptions yet.
- For the patient summary, group by `patient_id` and use aggregation functions. To count active prescriptions, filter where `end_date.isNull()` before counting.
- To create a comma-separated list of diagnosis codes, use `concat_ws(", ", collect_set("diagnosis_code"))`.
- Handle the physician null case BEFORE trying to extract nested fields, or you will get null pointer issues.

### Level 3 -- Advanced Hints (Code Snippets)

#### Flattening nested JSON:

```
diagnoses_flat = diagnoses_df.select(
    col("diagnosis_id"),
    col("patient_id"),
    col("diagnosis_code"),
    col("description"),
    col("diagnosis_date"),
    col("physician.id").alias("physician_id"),
    col("physician.name").alias("physician_name"),
    col("physician.specialty").alias("physician_specialty")
)
```

#### PII Masking:

```
from pyspark.sql.functions import sha2, col

patients_masked = patients_df \
    .withColumn("ssn_masked", sha2(col("ssn"), 256)) \
    .withColumn("first_name_masked", sha2(col("first_name"), 256)) \
    .withColumn("last_name_masked", sha2(col("last_name"), 256)) \
    .drop("ssn", "first_name", "last_name")
```

#### Patient Summary Aggregation:

```
from pyspark.sql.functions import count, countDistinct, collect_set, concat_ws, max, when

patient_summary = joined_df.groupBy("patient_id").agg(
    countDistinct("diagnosis_id").alias("diagnosis_count"),
    count(when(col("end_date").isNull(), 1)).alias("active_prescription_count"),
    countDistinct("prescription_id").alias("total_prescription_count"),
    concat_ws(", ", collect_set("diagnosis_code")).alias("diagnosis_codes"),
    concat_ws(", ", collect_set("drug_name")).alias("drug_names"),
    max("diagnosis_date").alias("latest_diagnosis_date"),
    max("start_date").alias("latest_prescription_date")
)
```

---

## 9. Validation Queries

---

Run these queries in Snowflake after loading data to verify correctness.

```
-- 1. Verify record counts in each layer
SELECT 'raw_patients' AS table_name, COUNT(*) AS row_count FROM RAW.raw_patients
UNION ALL
SELECT 'raw_diagnoses', COUNT(*) FROM RAW.raw_diagnoses
UNION ALL
SELECT 'raw_prescriptions', COUNT(*) FROM RAW.raw_prescriptions
UNION ALL
SELECT 'stg_patients', COUNT(*) FROM STAGING.stg_patients
UNION ALL
SELECT 'stg_diagnoses', COUNT(*) FROM STAGING.stg_diagnoses
UNION ALL
SELECT 'stg_prescriptions', COUNT(*) FROM STAGING.stg_prescriptions
UNION ALL
SELECT 'patient_360', COUNT(*) FROM CURATED.patient_360
UNION ALL
SELECT 'patient_summary', COUNT(*) FROM CURATED.patient_summary;

-- 2. Verify PII masking (SSN should be 64-character hex strings)
SELECT patient_id, ssn_masked, LENGTH(ssn_masked) AS hash_length
FROM STAGING.stg_patients
LIMIT 5;

-- 3. Verify no raw SSN values remain in staging/curated
SELECT COUNT(*) AS exposed_ssn_count
FROM STAGING.stg_patients
WHERE ssn_masked LIKE '____-__-____';
-- Expected: 0

-- 4. Verify physician fields are flattened
SELECT diagnosis_id, physician_id, physician_name, physician_specialty
FROM STAGING.stg_diagnoses
LIMIT 5;

-- 5. Verify null handling in diagnoses
SELECT COUNT(*) AS null_date_count
FROM STAGING.stg_diagnoses
WHERE is_date_missing = TRUE;
-- Expected: count of originally null diagnosis_date records

-- 6. Verify patient summary has one row per patient
SELECT COUNT(*) AS total_patients, COUNT(DISTINCT patient_id) AS distinct_patients
FROM CURATED.patient_summary;
-- Expected: both values should be 80

-- 7. Verify diagnosis counts make sense
SELECT patient_id, diagnosis_count, active_prescription_count
FROM CURATED.patient_summary
WHERE diagnosis_count > 2
ORDER BY diagnosis_count DESC;

-- 8. Verify no duplicate patient records in staging
SELECT patient_id, COUNT(*) AS cnt
FROM STAGING.stg_patients
GROUP BY patient_id
HAVING cnt > 1;
-- Expected: 0 rows

-- 9. Check active vs completed prescriptions
SELECT
    COUNT(CASE WHEN end_date IS NULL THEN 1 END) AS active_prescriptions,
    COUNT(CASE WHEN end_date IS NOT NULL THEN 1 END) AS completed_prescriptions
FROM STAGING.stg_prescriptions;

-- 10. Cross-check: patients with diagnoses but no prescriptions
SELECT ps.patient_id, ps.diagnosis_count, ps.total_prescription_count
FROM CURATED.patient_summary ps
WHERE ps.diagnosis_count > 0 AND ps.total_prescription_count = 0;
```

---

## 10. Submission Checklist

---

- ☐ Snowflake database and schemas created
- ☐ All three source files read successfully in PySpark
- ☐ Nested physician JSON flattened into flat columns
- ☐ Null values handled appropriately per field
- ☐ Duplicate records removed
- ☐ SSN and patient names masked with SHA-256
- ☐ All three datasets joined into patient\_360 view
- ☐ Patient summary aggregation created
- ☐ Glue bookmarks configured for incremental loads
- ☐ Data loaded to all three Snowflake layers (Raw, Staging, Curated)
- ☐ All validation queries pass

---

**Estimated Completion Time:** 3-4 hours

**Difficulty Level:** Intermediate

**Prerequisites:** Basic PySpark, SQL fundamentals, understanding of data warehousing concepts